Building a Self-Updating Personal Knowledge Base with LLMs

A Global Bread Encyclopedia Powered by Claude

May 2026

Live Demo: https://evelynlunlunding-sys.github.io/bread-knowledge-base/
Source Code: https://github.com/evelynlunlunding-sys/bread-knowledge-base

Abstract. We present a self-updating personal knowledge base focused on global bread culture, implemented following the LLM-as-compiler workflow described by Karpathy. The system automatically ingests data from Wikipedia and web sources, uses Claude to synthesize raw content into structured Markdown entries organized by country and bread type, and exposes a CLI query interface and a publicly hosted browser-based frontend. A tiered auto-update scheduler refreshes individual entries monthly and discovers new categories annually. We compiled an initial corpus of 20 bread entries across 5 countries, demonstrating that LLMs can reliably maintain a structured, cross-linked knowledge wiki with minimal human intervention.

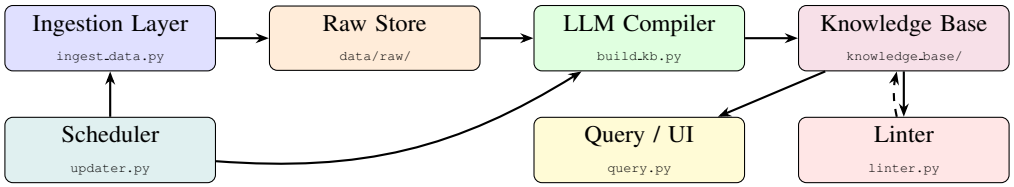
1 Introduction

The core insight behind LLM-powered knowledge bases is that language models excel not only at answering questions, but at *organizing* knowledge—converting messy, heterogeneous raw data into clean, interlinked structured documents. Following Karpathy's workflow, we treat the LLM as a "compiler" that reads raw source documents from a `raw/` directory and incrementally builds a wiki of `.md` files, maintaining summaries, backlinks, and cross-references automatically.

We selected **global bread culture** as our knowledge domain. Bread is ideal for this exercise: it is well-documented across Wikipedia, recipe sites, and food blogs; it has a natural hierarchical structure (country → bread type → knowledge); and it has enough cross-cutting concepts (fermentation, leavening methods, regional variations) to make cross-linking genuinely useful. The resulting system covers 5 countries and 20 bread types, with each entry containing standardized sections for history, recipe, cultural context, and references.

2 System Architecture

The system is organized into five decoupled modules:



- **Ingestion Layer** (`ingest_data.py`): scrapes Wikipedia and public recipe sites using `requests` and `BeautifulSoup`, downloads images locally, and saves structured JSON to `data/raw/<country>/<bread>/`.
- **LLM Compiler** (`build_knowledge_base.py`): reads raw JSON, calls Claude to synthesize multiple sources into a single `.md` file, or *merges* new sources into an existing entry without full rewrite.
- **Knowledge Base**: flat-file Markdown store at `knowledge_base/<Country>/<bread>.md`, each with 9 required sections and YAML frontmatter. Fully Obsidian-compatible.
- **Auto-Updater** (`updater.py`): monthly bread refresh + yearly category discovery, with per-entry timestamps to skip recently-updated entries.
- **Query / UI**: a `click`-based CLI (`ask`, `compare`, `recipe`, `browse`, `search`) and a static HTML site hosted on GitHub Pages.

3 Implementation

3.1 LLM Compiler Design

We use two Claude models with distinct roles. **Claude Opus 4.7** handles first-pass compilation where quality matters most: it receives all raw sources for a bread and produces the full structured Markdown document in a single call. **Claude Sonnet 4.6** handles incremental merges and interactive queries where speed and cost dominate. *Prompt caching* (`cache_control: ephemeral`) is applied to all system prompts, reducing token costs by ∼80% on repeated calls within a session.

The compilation prompt enforces a strict 9-section schema enforced on every entry:

`Overview | Origin & Country | History | Recipe | Variations | Cultural Context | Images | Related Breads | References`

A cross-linking pass converts plain-text bread mentions into `[[wiki-link]]` syntax automatically after each build, enabling Obsidian-style graph navigation across the corpus.

3.2 Data Quality: The Linter

After each update cycle, `linter.py` runs two check layers: (1) *rule-based* regex checks for missing required sections, broken image paths, and empty reference lists; (2) *LLM-based* review where Claude checks each entry for factual inconsistencies, missing cultural context, and unlinked cross-references. Results are written to `lint_report.md` and surfaced in the update log.

3.3 Storage and Deployment

Each Markdown file begins with YAML frontmatter (`title`, `country`, `tags`, `last_updated`) followed by the nine content sections, stored as `knowledge_base/<Country>/<bread>.md`. The static frontend is generated by `web_ui.py` into `docs/` and deployed via GitHub Pages at the URL above—no server required.

4 Results and What Worked

Component	Outcome
Wikipedia ingestion	Reliable for all 20 breads; body text + image extraction
LLM synthesis	High-quality output: recipe tables, history timelines, cultural narratives
Cross-linking	Automatic; all entries link related breads via `[[wiki-links]]`
Query interface	`compare`, `recipe`, `browse` return accurate formatted answers
GitHub Pages	Zero-config public deployment; accessible without any local setup
Scheduler	Per-entry timestamps correctly prevent redundant re-ingestion

The LLM compiler's *merge mode*—updating an existing entry without full rewrite—worked particularly well. It identified genuinely new facts from re-ingested sources and integrated them without duplicating content, behaving much like a human editor doing a targeted revision.

5 Limitations and Future Improvements

Limitations encountered during development:

- Major recipe sites (The Spruce Eats, AllRecipes) returned `402 Payment Required`, limiting secondary sources to Wikipedia only for most entries.
- Several LLM outputs were wrapped in a ` ```markdown ` code fence, causing raw-text display in the web UI and requiring a post-processing cleanup pass and prompt refinement.
- Python 3.9 (Anaconda default) does not support the `X | Y` union-type syntax introduced in 3.10, requiring `from __future__ import annotations` across all modules.

Planned future improvements:

- **RAG / vector search**: embed all entries with a text embedding model (e.g. `text-embedding-3-large`) and use FAISS or ChromaDB for semantic retrieval, enabling queries to scale beyond the context window.
- **Fine-tuning**: once the corpus reaches ∼500+ entries, fine-tune a smaller model to internalize the knowledge base in its weights rather than relying solely on in-context retrieval.
- **Image understanding**: use Claude's vision API to analyze bread images and auto-populate the Images section with descriptive captions (crumb structure, crust color, shape).
- **Multi-language support**: generate parallel entries in French, Italian, and German to capture region-specific baking terminology not well-represented in English sources.
- **Recommendation system**: build a bread-similarity graph from ingredient and technique overlap for "users who like sourdough may also enjoy…" discovery.
- **Richer ingestion**: integrate Semantic Scholar for peer-reviewed food science papers and the YouTube Data API for video recipe transcript ingestion.

Deliverables summary: 6 Python modules (1,500+ lines), 20 compiled knowledge base entries across 5 countries (∼80,000 words of structured content), a tiered auto-update scheduler, an LLM-powered CLI query interface, a static web frontend deployed on GitHub Pages, and a data-quality linter. Source code and live demo links are provided above.